COLEGIO UNIVERSITARIO DE CARTAGO

PROGRAMACIÓN V

TAREA CORTA 3

Investigación: Consumo de APIs REST

Servicios web, métodos HTTP, Fetch, Axios, JSON y renderizado de datos

Estudiante	Alejandro Morales Rojas
Responsabilidad	Persona 3 – Consumo de APIs REST
Proyecto	Aplicación React Native con Expo
API utilizada	DEV.to / Forem API v1
Fecha	5 de noviembre de 2026

Documento correspondiente exclusivamente a la investigación asignada a la Persona 3.

Índice

- Introducción
- 1. ¿Qué es una API REST y cómo funciona?
- 2. Métodos HTTP, principalmente GET
- 3. Uso de fetch() y Axios
- 4. Manejo e interpretación de respuestas JSON
- 5. Renderizado de información obtenida de una API
- 6. Aplicación de los conceptos en el proyecto
- Conclusiones
- Referencias bibliográficas

Alcance. Esta investigación desarrolla los cinco temas asignados a la Persona 3 en la distribución de trabajo de la Tarea Corta 3. La explicación práctica se limita al consumo de datos REST y su conexión con la interfaz desarrollada por la Persona 2.

Introducción

Las aplicaciones móviles suelen necesitar información que no se encuentra almacenada directamente en el dispositivo. Para obtenerla, pueden comunicarse con servicios externos mediante interfaces de programación de aplicaciones (API). Una API REST permite solicitar y manipular recursos a través de HTTP siguiendo convenciones que facilitan la comunicación entre sistemas.

En esta investigación se explican los fundamentos de las API REST, los métodos HTTP, el consumo de servicios con fetch() y Axios, la interpretación de JSON y la presentación de datos en React Native. Los conceptos se relacionan con el desarrollo práctico realizado para la Tarea Corta 3: una aplicación con Expo que consulta artículos públicos de DEV.to y los presenta en un catálogo.

El objetivo no es únicamente describir las herramientas, sino comprender el recorrido de los datos: el cliente realiza una solicitud, el servidor responde, la aplicación interpreta la información, administra sus estados y finalmente representa los resultados en la interfaz.

1. ¿Qué es una API REST y cómo funciona?

1.1 Concepto de API

Una API (Application Programming Interface o interfaz de programación de aplicaciones) es un conjunto de reglas y mecanismos mediante los cuales un programa puede solicitar servicios o datos a otro programa. La API define qué operaciones están disponibles, qué información debe enviarse y qué tipo de respuesta puede recibirse.

Por ejemplo, una aplicación de catálogo no necesita conocer cómo DEV.to almacena internamente sus artículos. En su lugar, solicita los artículos a través de un endpoint documentado y recibe una representación de esos recursos.

1.2 Concepto de REST

REST (Representational State Transfer) es un estilo arquitectónico para diseñar sistemas distribuidos. En una API que sigue este estilo, la información se organiza como recursos identificables mediante URI o direcciones. El cliente interactúa con esos recursos utilizando operaciones HTTP y recibe representaciones, comúnmente en formato JSON.

REST no es un lenguaje de programación ni una biblioteca. Es un conjunto de restricciones de diseño. Entre las ideas centrales se encuentran la separación cliente-servidor, la comunicación sin estado (stateless), una interfaz uniforme y la posibilidad de aprovechar mecanismos de caché cuando corresponda.

- **Cliente-servidor:** la interfaz de usuario y el servidor tienen responsabilidades separadas. El cliente solicita información; el servidor procesa la solicitud y responde.
- **Sin estado:** cada solicitud debe contener la información necesaria para ser entendida. El servidor no debería depender de recordar el contexto de solicitudes anteriores para interpretar la petición actual.
- **Interfaz uniforme:** los recursos se identifican de forma consistente y se manipulan mediante operaciones conocidas.
- **Representaciones:** el servidor entrega una representación del recurso, por ejemplo un objeto JSON.
- **Caché:** ciertas respuestas pueden almacenarse temporalmente para reducir solicitudes y mejorar el rendimiento, de acuerdo con las reglas de caché HTTP.

1.3 Recursos, URI y endpoints

Un recurso es cualquier elemento que el servicio expone, como un artículo, usuario, producto o comentario. Una URI identifica dónde se encuentra ese recurso. En el uso cotidiano, suele llamarse endpoint a la dirección concreta que el cliente consulta para ejecutar una operación.

```
GET https://dev.to/api/articles?tag=reactnative&per_page=30
```

En este endpoint se distinguen tres partes:

- **Servidor:** https://dev.to
- **Ruta:** /api/articles, que identifica la colección de artículos.
- **Parámetros de consulta:** tag=reactnative filtra por la etiqueta indicada y per_page=30 solicita hasta 30 elementos por página.

La documentación de Forem indica que GET /api/articles recupera artículos publicados, admite filtrado por etiqueta mediante tag y utiliza 30 elementos por página de forma predeterminada. El parámetro per_page permite definir el tamaño de página dentro de los límites documentados (Forem, s. f.).

1.4 Flujo general de una API REST

- **1. El cliente prepara la solicitud:** define el endpoint, el método HTTP y, si corresponde, encabezados o cuerpo.
- **2. La solicitud viaja al servidor:** se transmite mediante HTTP o HTTPS.
- **3. El servidor procesa la petición:** valida los parámetros, consulta sus recursos y determina el resultado.
- **4. El servidor responde:** devuelve un código de estado, encabezados y, normalmente, un cuerpo con datos.
- **5. El cliente interpreta la respuesta:** comprueba el estado, convierte el cuerpo al formato esperado y actualiza la interfaz.

La separación permite que una misma API sea utilizada por diferentes clientes, por ejemplo una aplicación móvil, una página web u otro servicio.

2. Métodos HTTP, principalmente GET

HTTP define métodos que expresan la acción que el cliente desea realizar sobre un recurso. En las API REST, los métodos más habituales son GET, POST, PUT, PATCH y DELETE. La operación concreta que permite cada endpoint depende del contrato publicado por el servidor.

Método	Propósito habitual	Ejemplo conceptual
GET	Consultar o recuperar un recurso. No se utiliza para solicitar cambios en el estado del servidor.	GET /api/articles
POST	Enviar datos para crear un recurso o iniciar un procesamiento.	POST /api/articles
PUT	Crear o reemplazar la representación de un recurso en una URI determinada.	PUT /api/users/15
PATCH	Aplicar una modificación parcial a un recurso.	PATCH /api/users/15
DELETE	Solicitar la eliminación de un recurso.	DELETE /api/users/15

GET es el método principal en esta implementación porque el catálogo necesita consultar artículos publicados, no crear ni modificar contenido en DEV.to. GET se considera un método seguro: su semántica está orientada a recuperar información y no a producir cambios solicitados en el recurso (MDN, s. f.).

2.1 Profundización en GET

Una solicitud GET identifica el recurso que se desea consultar. Los criterios de búsqueda o filtros pueden enviarse como parámetros de consulta en la URL. En el proyecto se utiliza la etiqueta reactnative y se solicita un máximo de 30 artículos.

```
const response = await fetch(
  'https://dev.to/api/articles?tag=reactnative&per_page=30',
  {
    method: 'GET',
    headers: {
      Accept: 'application/vnd.forem.api-v1+json'
    }
  }
);
```

El encabezado Accept expresa el formato de representación que el cliente espera recibir. La documentación de Forem recomienda el valor application/vnd.forem.api-v1+json para utilizar la versión 1 de su API. El endpoint público de artículos no requiere una clave API para esta consulta pública documentada (Forem, s. f.).

GET normalmente no lleva un cuerpo de solicitud para transmitir los criterios de consulta; estos se expresan en la URL y sus parámetros. La respuesta puede contener una colección JSON con los artículos que cumplen el filtro.

2.2 Códigos de estado HTTP relevantes

El código de estado permite conocer el resultado general de la solicitud. Los códigos se agrupan en cinco clases: 1xx informativos, 2xx satisfactorios, 3xx redirecciones, 4xx errores asociados a la solicitud del cliente y 5xx errores del servidor (MDN, s. f.).

Código	Significado	Interpretación práctica
200 OK	Solicitud exitosa.	Los artículos se recibieron correctamente.
201 Created	Recurso creado.	Respuesta frecuente después de una creación mediante POST.
204 No Content	Éxito sin cuerpo de respuesta.	La operación terminó, pero no hay contenido que mostrar.
400 Bad Request	Solicitud incorrecta.	Los parámetros enviados no cumplen lo esperado.
401 Unauthorized	Falta autenticación válida.	El servicio requiere credenciales.
403 Forbidden	Acceso denegado.	El cliente no tiene permiso para el recurso.
404 Not Found	Recurso no encontrado.	La ruta o el recurso solicitado no existe.
429 Too Many Requests	Demasiadas solicitudes.	El servidor limita temporalmente las peticiones.
500 Internal Server Error	Error interno del servidor.	El servicio encontró una condición inesperada.
503 Service Unavailable	Servicio no disponible.	El servidor no puede atender temporalmente la solicitud.

En las verificaciones del proyecto se observó una respuesta HTTP 429 durante consultas repetidas realizadas en la exploración. Este código no indica necesariamente un error del código cliente: informa que se superó el límite de solicitudes permitido durante un intervalo. Por ello, la aplicación debe mostrar un mensaje comprensible y evitar peticiones innecesarias.

3. Uso de fetch() y Axios

3.1 Fetch API

Fetch es una interfaz de JavaScript para realizar solicitudes de red y procesar sus respuestas. La función fetch() devuelve una Promesa (Promise), que permite trabajar de forma asíncrona. Al completarse la respuesta inicial, se obtiene un objeto Response con información como el estado HTTP, los encabezados y métodos para leer el cuerpo (MDN, s. f.).

Fetch está disponible como capacidad de la plataforma y no requiere instalar una dependencia adicional. En este proyecto se utiliza fetch() directamente, por lo que el consumo HTTP permanece separado de la interfaz y no se agrega una biblioteca externa.

3.2 Ejemplo básico con fetch()

```
async function obtenerArticulos() {
  try {
    const response = await fetch(
      'https://dev.to/api/articles?tag=reactnative&per_page=30'
    );

    if (!response.ok) {
      throw new Error(`Error HTTP: ${response.status}`);
    }

    const articulos = await response.json();
    return articulos;
  } catch (error) {
    console.error('No fue posible obtener los artículos:', error);
    throw error;
  }
}
```

La palabra async permite utilizar await dentro de la función. await espera la resolución de la Promesa sin bloquear la ejecución general de la aplicación. La propiedad response.ok es verdadera cuando el estado está en el rango 200–299. Es importante comprobarla porque fetch() no rechaza automáticamente la Promesa ante respuestas HTTP como 404 o 500; esos casos se reciben como objetos Response y deben validarse expresamente (MDN, s. f.).

- **response.status:** devuelve el código numérico de estado HTTP, por ejemplo 200, 404 o 429.
- **response.ok:** indica si el estado está dentro del rango 200–299.
- **response.json():** lee el cuerpo y lo interpreta como JSON. Es asíncrono y devuelve una Promesa.

El bloque try/catch permite manejar errores de red, errores lanzados al detectar estados HTTP no satisfactorios y fallos al interpretar el cuerpo. Una respuesta HTTP de error y una falla de conexión son situaciones distintas y conviene diferenciarlas.

3.3 ¿Qué es Axios?

Axios es una biblioteca de JavaScript basada en Promesas para realizar solicitudes HTTP desde aplicaciones web y entornos compatibles con Node.js. Ofrece una API de alto nivel para configurar solicitudes, transformar datos, establecer valores predeterminados, utilizar interceptores y gestionar errores.

```
import axios from 'axios';

async function obtenerArticulos() {
```

```
  const response = await axios.get(
    'https://dev.to/api/articles',
    {
      params: {
        tag: 'reactnative',
        per_page: 30
      },
      headers: {
        Accept: 'application/vnd.forem.api-v1+json'
      }
    }
  );

  return response.data;
}
```

En Axios, los parámetros de consulta pueden declararse mediante params y el contenido de la respuesta se encuentra normalmente en response.data. Axios también rechaza la Promesa por defecto ante respuestas que no cumplen su configuración de estados aceptados, lo que permite capturar muchos errores HTTP directamente en catch.

3.4 Comparación entre Fetch y Axios

Aspecto	Fetch	Axios
Disponibilidad	API integrada en plataformas compatibles; no requiere instalación adicional.	Biblioteca externa; debe agregarse al proyecto.
Lectura JSON	Se llama explícitamente response.json().	La respuesta JSON se transforma automáticamente en response.data en los casos habituales.
Errores HTTP	Se debe comprobar response.ok o response.status.	Rechaza por defecto estados fuera del rango aceptado.
Parámetros URL	Se pueden construir en la URL o con URLSearchParams.	Admite la opción params.
Configuración común	Opciones de fetch y objetos Request/Headers.	Instancias, configuración predeterminada e interceptores.
Uso en el proyecto	Es la opción seleccionada.	Se estudia como alternativa, pero no se instaló.

No existe una opción universalmente superior. Fetch es suficiente para solicitudes sencillas y reduce dependencias. Axios puede resultar conveniente cuando una aplicación necesita configuración compartida, interceptores o una API de consumo más uniforme. Para el catálogo de esta tarea, fetch() cubre los requisitos y mantiene el proyecto ligero.

4. Manejo e interpretación de respuestas JSON

4.1 ¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato de intercambio de datos basado en texto. Se utiliza para representar información estructurada que puede transmitirse entre sistemas. Aunque su sintaxis se parece a la de los objetos de JavaScript, JSON es un formato independiente y tiene reglas propias (MDN, s. f.).

JSON admite cadenas, números, valores booleanos, null, arreglos y objetos. Los nombres de las propiedades deben escribirse como cadenas entre comillas dobles. No admite directamente funciones, comentarios ni valores undefined.

```
{
  "id": 123,
  "title": "Introducción a React Native",
```

```
  "description": "Conceptos para desarrollar aplicaciones móviles",
  "tag_list": ["reactnative", "javascript"],
  "reading_time_minutes": 5,
  "published": true,
  "cover_image": null
}
```

El ejemplo representa un artículo. Las llaves delimitan un objeto; cada propiedad tiene un nombre y un valor. tag_list es un arreglo de cadenas, reading_time_minutes es un número, published es booleano y cover_image contiene null para indicar ausencia de valor.

4.2 Interpretar JSON recibido

Cuando fetch() recibe una respuesta, response.json() lee el cuerpo y analiza el texto JSON para producir valores de JavaScript que el programa puede utilizar. Si el cuerpo no contiene JSON válido, la operación puede fallar y debe manejarse la excepción.

```
const response = await fetch(url);

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const data = await response.json();

if (!Array.isArray(data)) {
  throw new Error('La respuesta no tiene el formato esperado');
}
```

En el caso del catálogo, la respuesta esperada es un arreglo de artículos. Por eso, además de validar que la solicitud haya sido exitosa y que el JSON pueda analizarse, el servicio comprueba Array.isArray(payload). Esta validación evita tratar una estructura inesperada como si fuera una lista.

4.3 Validaciones necesarias

- **Estado HTTP:** verificar si la solicitud fue exitosa antes de procesar el cuerpo.
- **Formato JSON:** controlar errores de análisis cuando el cuerpo no sea JSON válido.
- **Estructura:** comprobar que el tipo de dato recibido coincida con lo esperado, por ejemplo un arreglo.
- **Campos opcionales:** no asumir que todos los artículos contienen imagen, descripción u otros campos.
- **Lista vacía:** un arreglo vacío puede ser una respuesta válida; no debe confundirse con un error.

La validación estructural es importante porque una respuesta puede ser JSON válido y aun así no contener la estructura que la aplicación necesita. Por ejemplo, un objeto JSON no es equivalente a un arreglo JSON.

5. Renderizado de información obtenida de una API

5.1 El estado en React

Una vez recibidos los datos, la interfaz debe conservarlos y actualizarse cuando cambien. React ofrece Hooks para administrar estado y sincronizar componentes con sistemas externos. useState permite declarar una variable de estado y una función para actualizarla; useEffect permite ejecutar efectos secundarios, como una solicitud de red, cuando el componente se monta o cuando cambian sus dependencias.

5.2 useState

```
const [apiData, setApiData] = useState([]);
const [apiLoading, setApiLoading] = useState(true);
const [apiError, setApiError] = useState(null);
```

En este ejemplo se mantienen tres estados independientes:

- **apiData:** contiene los artículos obtenidos.

- **apiLoading:** indica si la solicitud sigue en proceso.
- **apiError:** contiene un mensaje cuando ocurre un problema.

Cuando se llama a la función actualizadora, React programa una nueva representación del componente con el valor actualizado. De esta forma, los datos recibidos pueden reflejarse en la pantalla sin manipular manualmente el árbol visual.

5.3 useEffect para iniciar la solicitud

```
useEffect(() => {
  const controller = new AbortController();

  async function cargar() {
    setApiLoading(true);
    setApiError(null);

    try {
      const articulos = await fetchCatalogItems({
        signal: controller.signal
      });
      setApiData(articulos);
    } catch (error) {
      if (error.name !== 'AbortError') {
        setApiData([]);
        setApiError('No se pudieron cargar los artículos.');
      }
    } finally {
      setApiLoading(false);
    }
  }

  cargar();

  return () => controller.abort();
}, []);
```

El arreglo vacío de dependencias indica que el efecto no depende de valores reactivos externos y se ejecuta al establecerse el componente. El efecto devuelve una función de limpieza que cancela la solicitud si deja de ser necesaria. React recomienda utilizar la limpieza de los efectos para evitar que una operación externa permanezca activa cuando cambia el ciclo de vida del componente (React, s. f.).

El ejemplo es ilustrativo: el proyecto real separa esta lógica en el Hook useCatalogItems.js y el servicio catalogApi.js.

5.4 Estados de carga, error y lista vacía

La interfaz debe representar de manera distinta las situaciones que puede experimentar el usuario:

Estado	Condición	Respuesta visual esperada
Cargando	La solicitud todavía no termina.	Indicador de actividad y texto de carga.
Éxito con datos	La solicitud terminó y el arreglo contiene artículos.	Listado de tarjetas.
Éxito sin datos	La solicitud terminó correctamente y el arreglo está vacío.	Mensaje de lista vacía.
Error	Falló la conexión, el servidor respondió con error o los datos no pudieron procesarse.	Mensaje de error comprensible.

Distinguir estos estados evita mensajes engañosos. Una lista vacía no necesariamente significa que hubo un error; puede ser el resultado correcto de una consulta sin coincidencias. Del mismo modo, un error de conexión no debe presentarse como si el servidor hubiera respondido con cero registros.

5.5 FlatList en React Native

FlatList es un componente de React Native diseñado para representar listas desplazables de manera eficiente. Recibe una colección mediante data y utiliza renderItem para indicar cómo representar cada elemento. También admite keyExtractor para obtener una clave estable, además de componentes para encabezados, separadores y contenido vacío.

```
<FlatList
  data={items}
  keyExtractor={(item) => String(item.id)}
  renderItem={({ item }) => (
    <ItemCard item={item} />
  )}
  ListEmptyComponent={renderEmptyList}
/>
```

En la aplicación, HomeScreen utiliza FlatList para presentar los artículos normalizados. Cada artículo se entrega al componente ItemCard, mientras que CategoryFilter permite seleccionar una categoría. FlatList facilita el renderizado de colecciones y evita tener que construir manualmente una lista de vistas.

La documentación de React Native señala que FlatList es un componente optimizado para listas planas y que keyExtractor permite definir la clave utilizada para identificar los elementos. También debe considerarse que FlatList es un componente de renderizado puro: los cambios en información externa que afecten la lista deben reflejarse en sus props (React Native, s. f.).

6. Aplicación de los conceptos en el proyecto

6.1 API seleccionada

Para el desarrollo práctico se seleccionó DEV.to, una plataforma de publicación de artículos tecnológicos que expone una API pública de Forem. La consulta implementada utiliza el endpoint GET /api/articles con los parámetros tag=reactnative y per_page=30. La documentación oficial describe este endpoint como una consulta de artículos publicados y documenta el filtrado por etiqueta y la paginación.

```
https://dev.to/api/articles?tag=reactnative&per_page=30
```

La aplicación envía el encabezado Accept con el tipo application/vnd.forem.api-v1+json, correspondiente a la versión 1 de la API de Forem. Durante la verificación funcional, la consulta devolvió 30 registros, que se mostraron en el catálogo.

6.2 Organización de responsabilidades por archivo

Archivo	Responsabilidad
catalogApi.js	Realiza la petición GET con fetch(), valida response.ok, procesa JSON, comprueba la estructura y clasifica errores HTTP, de conexión o formato.
useCatalogItems.js	Administra apiData, apiLoading y apiError mediante useState. Ejecuta la carga con useEffect y cancela la solicitud con AbortController cuando corresponde.
normalizeItem.js	Adapta los campos de los artículos de DEV.to al modelo requerido por la tarjeta: id, title, subtitle, description, image, badge y meta.
App.js	Ejecuta el Hook y entrega apiData, apiLoading y apiError a HomeScreen mediante props.
HomeScreen.js	Administra la presentación, aplica filtros por categoría y muestra estados de carga, error o lista vacía. Conserva los componentes visuales de Persona 2.

6.3 Recorrido de los datos

- **1. Solicitud:** App ejecuta useCatalogItems y el Hook inicia la carga.

- **2. Servicio:** fetchCatalogItems realiza la solicitud GET a DEV.to.
- **3. Respuesta:** el servicio comprueba el estado HTTP, interpreta el cuerpo como JSON y valida que sea un arreglo.
- **4. Estado:** el Hook guarda los artículos o el error y actualiza el indicador de carga.
- **5. Comunicación:** App pasa los estados a HomeScreen mediante props.
- **6. Adaptación:** HomeScreen aplica normalizeItem a cada artículo para obtener el modelo visual.
- **7. Renderizado:** FlatList representa cada elemento mediante ItemCard y permite utilizar los filtros de categoría.

6.4 Adaptación del modelo

La API entrega propiedades propias de DEV.to, por ejemplo title, description, cover_image, social_image, tag_list y reading_time_minutes. La interfaz existente no debe depender directamente de todas esas propiedades. Por eso normalizeItem funciona como adaptador: traduce la estructura externa a una estructura uniforme que los componentes visuales ya conocen.

```
// Ejemplo simplificado de adaptación
{
  id: String(rawItem.id),
  title: rawItem.title,
  subtitle: getArticleCategory(rawItem.tag_list),
  description: rawItem.description,
  image: rawItem.cover_image || rawItem.social_image,
  badge: rawItem.tag_list[0]?.toUpperCase(),
  meta: `Lectura: ${rawItem.reading_time_minutes} min`
}
```

La adaptación desacopla la interfaz del proveedor de datos. Si en el futuro se cambia la API, el trabajo principal consistiría en ajustar el servicio y el adaptador, procurando mantener el contrato que consume la pantalla.

6.5 Verificaciones prácticas realizadas

Durante la ejecución de la aplicación en Expo se observaron los siguientes resultados. Esta tabla registra únicamente las verificaciones comunicadas y observadas durante el desarrollo; no representa una certificación de pruebas automatizadas.

Verificación	Resultado observado
Solicitud GET y recepción de artículos	PASS: la aplicación mostró 30 registros de DEV.to.
Renderizado de artículos	PASS: se visualizaron títulos, descripciones, imágenes y tiempo de lectura.
Manejo de desconexión	PASS: al perder conexión se presentó un mensaje indicando que no fue posible conectar con la API.
Filtros por categoría	PASS: se confirmó el funcionamiento de los filtros.
Favoritos	PASS: se confirmó la interacción de marcar y desmarcar favoritos.
Exportación web de Expo	PASS: Codex reportó que la exportación web compiló correctamente.
Exportación Android de Expo	PASS: Codex reportó que la exportación Android compiló correctamente.

No se reporta como ejecutada una prueba controlada de respuesta vacía, ni una ejecución en dispositivo físico o emulador. Estas limitaciones deben mantenerse explícitas al presentar los resultados.

Conclusiones

Las API REST permiten que una aplicación móvil obtenga información de servicios externos mediante operaciones HTTP y recursos identificados por endpoints. Esta separación hace posible que el cliente se concentre en la experiencia de usuario mientras el servidor administra y proporciona los datos.

El método GET es apropiado cuando la aplicación necesita consultar información sin solicitar modificaciones en el servidor. Para consumirlo desde JavaScript, fetch() ofrece una alternativa integrada y basada en Promesas; Axios constituye otra opción mediante una biblioteca externa con utilidades adicionales.

JSON facilita el intercambio de información estructurada. Sin embargo, no basta con convertir una respuesta a JSON: también es necesario validar el estado HTTP, comprobar la estructura recibida y contemplar campos opcionales, respuestas vacías y errores.

En React Native, useState permite conservar los datos y estados de la solicitud, mientras que useEffect permite iniciar y limpiar el efecto asociado a la comunicación externa. FlatList facilita presentar colecciones de registros. La separación entre servicio, Hook, adaptador y pantalla mejora la organización y permite que la interfaz no dependa directamente del proveedor de datos.

En el proyecto de la Tarea Corta 3, estos conceptos se aplicaron mediante una consulta pública a DEV.to. Los artículos recibidos se adaptan al modelo visual existente y se muestran en el catálogo sin sustituir los componentes de Persona 2. Las verificaciones observadas confirmaron la carga de artículos, la presentación, los filtros, los favoritos y el manejo de desconexión.

Referencias bibliográficas

Forem. (s. f.). *Forem API V1 – Published articles*. https://developers.forem.com/api/v1

Forem. (s. f.). *API documentation*. https://developers.forem.com/api/

MDN Web Docs. (s. f.). *HTTP request methods*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

MDN Web Docs. (s. f.). *HTTP response status codes*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

MDN Web Docs. (s. f.). *Using the Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs. (s. f.). *JSON*. https://developer.mozilla.org/en-US/docs/Glossary/JSON

React. (s. f.). *useEffect*. https://react.dev/reference/react/useEffect

React. (s. f.). *useState*. https://react.dev/reference/react/useState

React Native. (s. f.). *FlatList*. https://reactnative.dev/docs/flatlist

Axios. (s. f.). *Getting started*. https://axios-http.com/docs/intro

Nota sobre las fuentes: las referencias se consultaron en documentación oficial o documentación técnica mantenida por los proyectos correspondientes. La fecha de consulta de esta investigación es 5 de noviembre de 2026.